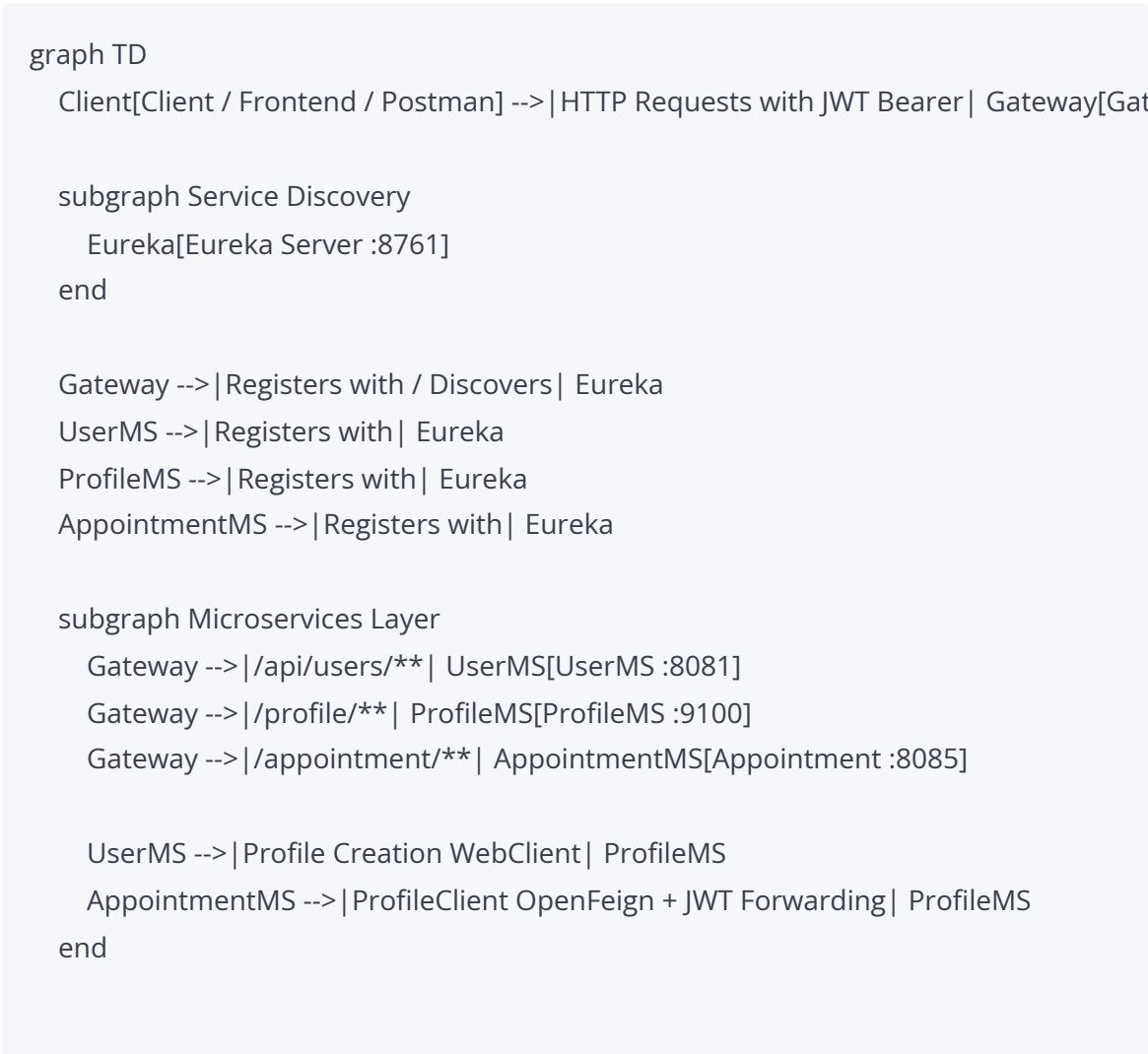


Hospital Management System (HMS) — System Specification Document (Version 1.0)

1. System Overview & Architecture

The **Hospital Management System (HMS)** is built on a modern Spring Boot microservices architecture. It provides secure, decoupled, and scalable management of medical staff, patients, and clinical appointments.

1.1 Architecture Diagram



```
subgraph Persistence Layer
  UserMS --> DB1[(MySQL: userdb)]
  ProfileMS --> DB2[(MySQL: profiledb)]
  AppointmentMS --> DB3[(MySQL: appointmentdb)]
end
```

2. Microservices Breakdown

Service	Port	Database	Primary Responsibility
Eureka Server	8761	None	Service registry and discovery. Allows microservices to find each other dynamically by application name without hardcoded IPs.
GateWayMS	9000	None	Single entry point (reverse proxy). Intercepts all incoming client requests, verifies JWT token signatures, and routes traffic to downstream services.
UserMS	8081	userdb	Authentication & credentials. Handles user registration, BCrypt password hashing, JWT generation, and triggers initial profile creation.
ProfileMS	9100	profiledb	Medical profile data. Manages doctor specialties, departments, licenses, experience, and patient personal/medical records.
Appointment	8085	appointmentdb	Scheduling lifecycle. Handles booking, conflict detection, rescheduling, completions, cancellations, and aggregated appointment details via OpenFeign.

3. Security Architecture (JWT & RBAC)

The system operates on a **Stateless, Pure-JWT Bearer Token** security model:

- **Token Issuance:** Upon successful authentication (`POST /api/users/login`), `UserMS` issues an HMAC-SHA256 signed JWT token valid for 5 hours.
 - **Token Payload:**
 - `sub` : User email / username
 - `id` : User database ID
 - `role` : User role (`PATIENT` , `DOCTOR` , or `ADMIN`)
 - `profileId` : Linked doctor or patient profile ID
 - `iat` & `exp` : Timestamp and expiration
 - **Gateway Token Filter:** `TokenGatewayFilterFactory` ensures protected routes contain a valid `Bearer` token before forwarding.
 - **Service-Level Filters:** Each microservice implements `JwtAuthenticationFilter` , which verifies the signature, parses roles into `ROLE_PATIENT` , `ROLE_DOCTOR` , or `ROLE_ADMIN` , and populates Spring's `SecurityContextHolder` .
 - **Inter-Service Communication:** Feign clients (`ProfileClient` in `Appointment`) use a `RequestInterceptor` that automatically extracts the incoming `Authorization: Bearer <token>` header and attaches it to downstream requests, ensuring unbroken security across service boundaries.
 - **Method-Level Security:** Endpoints are protected with `@EnableMethodSecurity` and `@PreAuthorize("hasRole(...)")` checks.
-

4. User Roles & Capabilities Matrix

The system defines three core roles: `PATIENT` , `DOCTOR` , and `ADMIN` .

```
classDiagram
class PATIENT {
    +Browse & Search Doctors
    +Filter Doctors by Department
    +Book Appointment
    +View My Appointments
    +Reschedule My Appointment
    +Cancel My Appointment
    +View & Update My Profile
}
class DOCTOR {
    +View Assigned Appointments Schedule
    +View Patient Details
    +Mark Appointment as COMPLETED
    +Reschedule Appointment
}
```

```

+Cancel Appointment
+Update Doctor Profile
+View All Patients
}
class ADMIN {
+Manage All Appointments
+Delete Doctor Profiles
+Delete Patient Records
+View System-wide Metrics & Schedules
+Update Any User Role or Details
}

```

Detailed Role Permissions Table

Action / Capability	PATIENT	DOCTOR	ADMIN	Endpoint
Register & Login				POST /api/users/register , POST /api/users/login
Browse / Search Doctors				GET /profile/doctor/all , GET /profile/doctor/department/{dept}
View Doctor Profile Details				GET /profile/doctor/get/{id}
Schedule Appointment				POST /appointment/schedule
View Own Appointments				GET /appointment/patient/{patientId}
View Doctor's Schedule				GET /appointment/doctor/{doctorId}
View Full Appointment Details				GET /appointment/get/details/{id}
Reschedule Appointment				PUT /appointment/reschedule/{id}

Complete Appointment				PUT /appointment/complete/{id}
Cancel Appointment				DELETE /appointment/cancel/{id}
Update Doctor Profile				PUT /profile/doctor/update
Update Patient Profile				PUT /profile/patient/update
View All Patients Directory				GET /profile/patient/all
Delete Doctor Profile				DELETE /profile/doctor/delete/{id}
Delete Patient Record				DELETE /profile/patient/delete/{id}

5. Complete API Reference

All requests route through Gateway: `http://localhost:9000`

5.1 Authentication (UserMS)

1. Register User

- **Method:** POST
- **Path:** /api/users/register
- **Access:** Public
- **Body:**

```
{
  "username": "Dr. Sarah",
  "email": "sarah@hospital.com",
  "password": "Password123!",
  "role": "DOCTOR"
}
```

- **Password Policy:** Minimum 8 characters, at least 1 uppercase, 1 lowercase, 1 digit, 1 special character.
- **Side-effect:** Creates `User` in `UserMS` and automatically provisions `Doctor` or `Patient` in `ProfileMS`.

2. Login

- **Method:** `POST`
- **Path:** `/api/users/login`
- **Access:** `Public`
- **Body:**

```
{
  "email": "sarah@hospital.com",
  "password": "Password123!"
}
```

- **Response:**

```
{
  "email": "sarah@hospital.com",
  "token": "eyJhbGciOiJIUzI1NiJ9..."
}
```

5.2 Profiles (`ProfileMS`)

[!NOTE]

All endpoints below require: `Authorization: Bearer <token>`

1. Get All Doctors

- **Method:** `GET`
- **Path:** `/profile/doctor/all`
- **Access:** `PATIENT` , `DOCTOR` , `ADMIN`

2. Filter Doctors by Department

- **Method:** `GET`
- **Path:** `/profile/doctor/department/{department}`
- **Access:** `PATIENT` , `DOCTOR` , `ADMIN`

3. Filter Doctors by Specialization

- **Method:** GET
- **Path:** /profile/doctor/specialization/{specialization}
- **Access:** PATIENT , DOCTOR , ADMIN

4. Get Doctor Profile by ID

- **Method:** GET
- **Path:** /profile/doctor/get/{id}
- **Access:** PATIENT , DOCTOR , ADMIN

5. Update Doctor Profile

- **Method:** PUT
- **Path:** /profile/doctor/update
- **Access:** DOCTOR , ADMIN

6. Delete Doctor Profile

- **Method:** DELETE
- **Path:** /profile/doctor/delete/{id}
- **Access:** ADMIN

7. Get All Patients

- **Method:** GET
- **Path:** /profile/patient/all
- **Access:** DOCTOR , ADMIN

8. Get Patient Profile by ID

- **Method:** GET
- **Path:** /profile/patient/get/{id}
- **Access:** PATIENT , DOCTOR , ADMIN

9. Update Patient Profile

- **Method:** PUT
- **Path:** /profile/patient/update
- **Access:** PATIENT , ADMIN

10. Delete Patient Profile

- **Method:** DELETE

- **Path:** /profile/patient/delete/{id}
 - **Access:** ADMIN
-

5.3 Appointments (Appointment)

[!NOTE]

All endpoints below require: Authorization: Bearer <token>

1. Schedule Appointment

- **Method:** POST
- **Path:** /appointment/schedule
- **Access:** PATIENT , ADMIN
- **Body:**

```
{
  "doctorId": 1,
  "patientId": 2,
  "appointmentTime": "2026-10-15T14:30:00",
  "reason": "Consultation",
  "notes": "Complaining of chest tightness"
}
```

- **Response:** Created appointmentId (Long).

2. Get Appointment Basic Summary

- **Method:** GET
- **Path:** /appointment/get/{appointmentId}
- **Access:** PATIENT , DOCTOR , ADMIN

3. Get Full Appointment Details (Aggregated with Doctor & Patient Info)

- **Method:** GET
- **Path:** /appointment/get/details/{appointmentId}
- **Access:** PATIENT , DOCTOR , ADMIN
- **Response:**

```
{
  "id": 1,
  "doctorId": 1,
```

```
"doctorName": "Dr. Sarah",
"doctorPhone": "01000000000",
"doctorEmail": "sarah@hospital.com",
"patientId": 2,
"patientName": "Ahmed Ali",
"patientPhone": "01100000000",
"patientEmail": "ahmed@example.com",
"appointmentTime": "2026-10-15T14:30:00",
"status": "SCHEDULED",
"reason": "Consultation",
"notes": "Complaining of chest tightness"
}
```

4. Get Patient's Appointments ("My Appointments")

- **Method:** GET
- **Path:** /appointment/patient/{patientId}
- **Access:** PATIENT , DOCTOR , ADMIN

5. Get Doctor's Schedule

- **Method:** GET
- **Path:** /appointment/doctor/{doctorId}
- **Access:** DOCTOR , ADMIN

6. Reschedule Appointment

- **Method:** PUT
- **Path:** /appointment/reschedule/{appointmentId}
- **Access:** PATIENT , DOCTOR , ADMIN
- **Body:**

```
{
  "newDateTime": "2026-10-20T11:00:00"
}
```

7. Mark Appointment as Completed

- **Method:** PUT
- **Path:** /appointment/complete/{appointmentId}
- **Access:** DOCTOR , ADMIN

8. Cancel Appointment

- **Method:** DELETE
 - **Path:** /appointment/cancel/{appointmentId}
 - **Access:** PATIENT , DOCTOR , ADMIN
-

6. Business Logic & Validation Rules

stateDiagram-v2

```
[*] --> SCHEDULED: scheduleAppointment()
SCHEDULED --> CANCELLED: cancelledAppointment()
SCHEDULED --> SCHEDULED: rescheduleAppointment()
SCHEDULED --> COMPLETED: completeAppointment()
CANCELLED --> [*]
COMPLETED --> [*]
```

1. Past Date Prevention:

- An appointment cannot be booked or rescheduled to a date/time prior to `LocalDateTime.now()` .
- Throws error: `INVALID_APPOINTMENT_TIME` .

2. Double-Booking Prevention:

- A doctor cannot have two appointments in `SCHEDULED` status at the identical `appointmentTime` .
- Throws error: `DOCTOR_UNAVAILABLE_SLOT` .

3. Status Immutability:

- Appointments marked `CANCELLED` cannot be rescheduled or marked `COMPLETED` (`APPOINTMENT_ALREADY_CANCELLED`).
- Appointments marked `COMPLETED` cannot be rescheduled or cancelled (`APPOINTMENT_ALREADY_COMPLETED`).

4. Referential Integrity via OpenFeign:

- Before saving an appointment, `Appointment` queries `ProfileMS` via Feign to confirm both `doctorExists` and `patientExists` . If non-existent, booking fails with `DOCTOR_NOT_FOUND` or `PATIENT_NOT_FOUND` .
-

7. Data Models & Database Schemas

7.1 `userdb` (`UserMS`)

- **users table:**
 - id (BIGINT, Primary Key, Auto Increment)
 - username (VARCHAR, Unique, Not Null)
 - email (VARCHAR, Unique, Not Null)
 - password (VARCHAR, BCrypt hash, Not Null)
 - role (VARCHAR: PATIENT , DOCTOR , ADMIN)
 - profile_id (BIGINT, Foreign Key to ProfileMS ID)

7.2 profiledb (ProfileMS)

- **doctors table:**
 - id (BIGINT, Primary Key)
 - username , email , phone , address , dob
 - license_no (VARCHAR, Unique)
 - blood_group (ENUM)
 - total_exp (INT)
 - department (VARCHAR)
 - specialization (VARCHAR)
- **patients table:**
 - id (BIGINT, Primary Key)
 - username , email , phone , address , dob
 - aadhar_no (VARCHAR, Unique)
 - blood_group (ENUM)

7.3 appointmentdb (Appointment)

- **appointments table:**
 - id (BIGINT, Primary Key)
 - doctor_id (BIGINT, Indexed)
 - patient_id (BIGINT, Indexed)
 - appointment_time (DATETIME, Indexed)
 - status (VARCHAR: SCHEDULED , CANCELLED , COMPLETED)
 - reason (VARCHAR)
 - notes (VARCHAR)
-

8. Deployment & Execution Instructions

Service Startup Order

1. Start Eureka Service Registry

```
cd "Eureka-Server" && ./mvnw spring-boot:run
```

2. Start Gateway

```
cd "GateWayMS" && ./mvnw spring-boot:run
```

3. Start Core Domain Services

```
cd "UserMS/UserMS" && ./mvnw spring-boot:run
```

```
cd "ProfileMS" && ./mvnw spring-boot:run
```

```
cd "Appointment" && ./mvnw spring-boot:run
```